Fundamentos de Java para principiantes — Una aventura en Java

Versión revisada: historia unificada

Público: estudiantes de secundaria de 12 a 17 años sin experiencia previa.

Formato: 10 videos cortos, aproximadamente 62 minutos.

Herramienta: OnlineGDB, Java en navegador; sin instalaciones.

Historia: Dan explora un pequeño mundo de aventuras, reúne puntos, supera retos y conoce mascotas virtuales. Todo sucede en la consola de texto; no se promete un juego gráfico.

Principios para grabar

- En todos los videos aparece el mismo universo: Dan, su energía, sus puntos, los retos y sus mascotas Luna, Rocky y Nube. Los ejemplos se ejecutan por separado en cada lección; no se supone que el código de un video se acumula automáticamente en el siguiente.
- Presentar una idea visual y cotidiana dentro del juego, mostrar un ejemplo corto, ejecutarlo, pedir una predicción y proponer una práctica sencilla.
- Mantener los términos de Java en pantalla, pero explicarlos antes de usarlos: `String` (texto), `int` (entero), `boolean` (verdadero/falso), etc.
- Mostrar el código en tamaño legible, con resaltados y una consola visible. Leer pausadamente los símbolos nuevos; no narrar cada carácter.
- Mostrar **una práctica principal** en el cierre. Poner desafíos opcionales y enlaces a ejemplos precargados en la descripción y en el plan de curso.
- Verificar que OnlineGDB funcione en el navegador sin instalación y que el enlace compartido abra el ejemplo correcto. En todos los archivos, la clase pública principal es `Main`.
- Subir los videos como **no listados**; no mostrar nombres, marcas ni logos de los autores. No solicitar cuentas, instalaciones ni pagos.
- La duración incluye demostraciones y breves silencios para pensar; las prácticas largas se hacen pausando el video.

Ruta de aprendizaje

Video	Tema	Duración	Hito de la aventura
1	¿Qué es programar y qué es Java?	5 min	La aventura muestra su bienvenida
2	Variables y tipos de datos	6 min	Dan obtiene una ficha de personaje
3	Operadores y expresiones	5 min	Calculamos puntos y energía
4	Decisiones con `if-else`	6 min	Dan decide si puede superar un obstáculo
5	Bucles `for` y `while`	6 min	Dan completa rondas y una cuenta regresiva
6	Métodos y parámetros	6 min	Creamos acciones que se reutilizan
7	Entrada y salida con `Scanner`	7 min	El jugador interactúa con un reto

8	Clases, objetos y constructores	7 min	Creamos mascotas virtuales
9	Encapsulación y arreglos	6 min	Protegemos y organizamos las mascotas
10	Proyecto final y buenas prácticas	8 min	Jugamos una aventura de texto completa
Total		**62 min**	

Video 1 — ¿Qué es programar y qué es Java? (5 min)

Objetivo: entender qué es una instrucción y ejecutar un primer programa en Java.

Idea clave: programar es indicar paso a paso qué debe hacer una computadora.

Guion por escenas

Escena 1 · El misterio de los videojuegos (0:00–0:40)

Pantalla: personaje Dan quieto en una escena sencilla de videojuego. Texto: «¿Cómo sabe qué hacer?».

Narración: «¿Alguna vez te preguntaste cómo sabe un personaje de videojuego cuándo avanzar, ganar puntos o perder una vida? No lo adivina. Detrás hay instrucciones que alguien escribió. Esas instrucciones forman un programa. Hoy vamos a crear nuestra primera instrucción para una pequeña aventura en Java. Solo necesitas un navegador».

Escena 2 · Dar instrucciones al personaje (0:40–1:40)

Pantalla: Dan frente a una puerta; aparecen tarjetas: «1. Avanzar», «2. Revisar si hay una llave», «3. Abrir la puerta si tiene llave». El personaje sigue las tarjetas en orden.

Narración: «Imagina que Dan está frente a una puerta. Si solo le decimos “¡gana el nivel!”, no sabrá qué hacer. Tenemos que darle pasos claros: avanzar, comprobar si tiene la llave y abrir la puerta cuando corresponda. Programar se parece a escribir las reglas de ese juego: la computadora sigue las instrucciones que le damos, en el orden y con las condiciones que indiquemos. Más adelante aprenderemos a escribir esas decisiones en Java Fíjate en que cada paso tiene una tarea concreta. Si falta la llave, la puerta no debería abrirse: más adelante aprenderemos a escribir esa regla con código.».

Escena 3 · ¿Por qué Java? (1:40–2:30)

Pantalla: «Muchos lenguajes, una misma idea: dar instrucciones»; aparece Java y una consola con mensajes del juego.

Narración: «Existen distintos lenguajes de programación. Java es uno de ellos y se utiliza para crear muchos tipos de aplicaciones. Lo usaremos porque permite aprender cómo organizar instrucciones, tomar decisiones y crear objetos. Por ahora, nuestra aventura será de texto: los mensajes aparecerán en la consola. No necesitamos crear gráficos para empezar a programar».

Escena 4 · La primera bienvenida (2:30–4:15)

Pantalla: abrir el ejemplo precargado de OnlineGDB; señalar editor, botón Run y consola; escribir y ejecutar.

Narración: «Este espacio es el editor, donde escribimos instrucciones. Al pulsar Run, veremos el resultado en la consola. Main es el nombre de nuestra clase principal; por ahora, piensa en ella como el lugar que contiene nuestro primer programa. main señala por dónde comienza la ejecución. System.out.println muestra un mensaje. El texto va entre comillas, las llaves agrupan instrucciones y el punto y coma termina esta instrucción. No necesitas memorizar todo hoy. Ejecutemos: ¡la aventura ya nos da la bienvenida! Observa también que Main, con mayúscula, y main, con minúscula, no son lo mismo: Main es la clase y main es el método desde el que comienza el programa. En System.out.println, println es la instrucción que muestra texto y después pasa a la siguiente línea. Las comillas indican exactamente qué mensaje queremos mostrar; si cambiamos ese texto, cambia lo que aparece en la consola.».

Escena 5 · Prueba y cierre (4:15–5:00)

Pantalla: «Pausa y prueba: cambia el nombre del héroe».

Narración: «Ahora te toca. Cambia “Dan” por tu nombre y vuelve a ejecutar. Si aparece tu mensaje, ¡lo lograste! En la próxima lección haremos que nuestro personaje tenga vidas, puntos y energía».

Código de demostración

```
public class Main {
    public static void main(String[] args) {
        System.out.println("¡Bienvenido a la aventura de Dan!");
        System.out.println("Tu misión comienza ahora.");
    }
}
```

Práctica principal: cambia Dan por el nombre de tu personaje.

Desafíos opcionales: agrega una tercera línea con el nombre del mundo; elimina temporalmente un punto y coma, observa el error y vuelve a colocarlo.

Video 2 — Variables y tipos de datos (6 min)

Objetivo: crear y modificar variables `String`, `int`, `double` y `boolean`.

Idea clave: una variable guarda un dato que el programa puede usar.

Guion por escenas

Escena 1 · La ficha del héroe (0:00–0:30)

Pantalla: ficha de Dan: nombre, vidas, velocidad, llave.

Narración: «En el video anterior, nuestra aventura saludó al jugador. Pero un héroe necesita más que un nombre: ¿cuántas vidas tiene?, ¿qué tan rápido se mueve?, ¿lleva una llave? Hoy vamos a guardar esos datos».

Escena 2 · Casillas de la ficha (0:30–1:30)

Pantalla: casillas con etiquetas `nombre`, `vidas`, `velocidad`, `tieneLlave`.

Narración: «Imagina que cada espacio de la ficha es una casilla con etiqueta. La etiqueta nos ayuda a encontrar un dato. En Java llamamos variable a un espacio con nombre donde guardamos un valor. Y cada variable tiene un tipo, según lo que necesitemos guardar Por ejemplo, vidas es el nombre de una variable y 3 es el valor que guarda. Podemos consultar ese valor y cambiarlo durante la partida.».

Escena 3 · Cuatro tipos (1:30–3:30)

Pantalla: escribir una línea a la vez y resaltar el resultado.

Narración: «String guarda texto, como “Dan”. int guarda números enteros, como tres vidas. double guarda números con decimales, como una velocidad de 1.5. boolean guarda true o false: por ejemplo, si Dan tiene una llave. Para crear una variable escribimos tipo, nombre y valor. En nombres como tieneLlave, la segunda palabra comienza con mayúscula; eso ayuda a leerlos En la línea int vidas = 3;, int indica el tipo, vidas es el nombre, el signo igual guarda el valor y el punto y coma cierra la instrucción. Las palabras true y false se escriben sin comillas porque representan valores de verdadero o falso, no textos.».

Escena 4 · Una vida menos (3:30–5:00)

Pantalla: resaltar `vidas = 2;` y la salida antes/después.

Narración: «Dan empezó con tres vidas, pero chocó con un obstáculo. Cambiamos el valor de vidas a dos. Antes de ejecutar, ¿cuántas vidas crees que mostrará ahora? [Pausa de 3 segundos]. Exacto: dos. El signo + también puede unir un texto con el valor de una variable para mostrarlo en pantalla Fíjate en la diferencia: al crearla escribimos int vidas = 3;, pero para cambiarla basta con vidas = 2;. No repetimos int porque la variable ya existe.».

Escena 5 · Práctica (5:00–6:00)

Pantalla: «Cambia las vidas y el nombre del héroe».

Narración: «Pausa el video, cambia el nombre del personaje y sus vidas, y comprueba la nueva ficha. En el siguiente video aprenderemos a calcular recompensas y energía».

Código de demostración

```
public class Main {
    public static void main(String[] args) {
        String nombre = "Dan";
        int vidas = 3;
        double velocidad = 1.5;
        boolean tieneLlave = false;
```

```
        System.out.println("Héroe: " + nombre);
        System.out.println("Vidas: " + vidas);
        System.out.println("Velocidad: " + velocidad);
        System.out.println("¿Tiene llave? " + tieneLlave);

        vidas = 2;
        System.out.println("Después del obstáculo, vidas: " + vidas);
    }
}
```

Práctica principal: cambia el nombre y las vidas; predice la salida.

Desafíos opcionales: cambia `tieneLlave` a `true`; crea `int puntos = 0;` y muéstralo; identifica por qué `int vidas = "tres";` no funciona.

Video 3 — Operadores y expresiones (5 min)

Objetivo: usar operadores aritméticos, comparaciones y operadores lógicos.

Idea clave: los operadores permiten calcular y comprobar condiciones del juego.

Guion por escenas

Escena 1 · La recompensa (0:00–0:30)

Pantalla: Dan encuentra dos cofres: uno con 10 monedas y otro con 5.

Narración: «Dan encontró dos cofres. Uno tiene diez monedas y el otro cinco. ¿Cuántas monedas consiguió en total? Para que el juego lo calcule, usaremos operadores».

Escena 2 · Calcular puntos y energía (0:30–2:00)

Pantalla: mostrar `+`, `-`, `*`, `/`, `%` asociados con monedas y energía.

Narración: «El signo más suma recompensas. El menos descuenta energía cuando Dan recibe daño. El asterisco multiplica, por ejemplo, las monedas de varias rondas. La barra divide. Y el porcentaje calcula el residuo: si repartimos siete monedas en grupos de dos, sobra una. Atención: en Java, al dividir dos enteros, 7 / 2 da 3; para obtener 3.5 usamos 7 / 2.0 En el código, el asterisco * significa multiplicar y la barra / significa dividir. El símbolo % no calcula porcentajes aquí: devuelve lo que sobra de una división.».

Escena 3 · Comparar (2:00–3:30)

Pantalla: «¿Tiene al menos 10 monedas?» y «¿Tiene llave y energía?».

Narración: «Los operadores de comparación responden verdadero o falso. Por ejemplo, monedas >= 10 pregunta si Dan tiene al menos diez monedas. == compara dos valores; un solo = asigna un valor. También podemos combinar preguntas: && significa “y”, y || significa “o”. No necesitas memorizar todos los símbolos: los iremos usando Por ejemplo, >= significa 'mayor o igual que'. Si monedas vale 15, monedas >= 10 será true. Cuando usamos &&, las dos condiciones deben cumplirse para obtener true.».

Escena 4 · Resolver el reto (3:30–4:30)

Pantalla: ejecutar el programa y mostrar monedas, energía y si puede abrir el cofre.

Narración: «Sumamos las monedas, restamos el daño y comprobamos si Dan tiene suficientes monedas y una llave. Antes de pulsar Run, ¿crees que podrá abrir el cofre? [Pausa]. Veamos la respuesta Observa los paréntesis: agrupan el cálculo o la comparación que queremos mostrar. Si el resultado es true, se cumplen las condiciones del cofre; si es false, falta algún requisito.».

Escena 5 · Práctica (4:30–5:00)

Pantalla: «Cambia las monedas del segundo cofre».

Narración: «Prueba cambiando cinco monedas por una. ¿La respuesta cambia? En la próxima lección haremos que el juego elija qué mensaje mostrar según la situación».

Código de demostración

```
public class Main {
    public static void main(String[] args) {
        int monedas = 10 + 5;
        int energia = 50 - 10;
        boolean tieneLlave = true;

        System.out.println("Monedas: " + monedas);
        System.out.println("Energía: " + energia);
        System.out.println("Recompensa triple: " + (5 * 3));
```

```
        System.out.println("7 / 2 = " + (7 / 2));
        System.out.println("7 / 2.0 = " + (7 / 2.0));
        System.out.println("Moneda sobrante: " + (7 % 2));
        System.out.println("¿Puede abrir el cofre? " +
                (monedas >= 10 && tieneLlave));
    }
}
```

Práctica principal: cambia las monedas iniciales y predice si puede abrir el cofre.

Desafíos opcionales: calcula la energía después de dos golpes de 5; prueba el operador `||`; calcula si un número de ronda es par con `% 2 == 0`.

Video 4 — Tomar decisiones con `if-else` (6 min)

Objetivo: escribir decisiones con `if`, `else if` y `else`.

Idea clave: el juego ejecuta acciones diferentes según una condición.

Guion por escenas

Escena 1 · La puerta del castillo (0:00–0:30)

Pantalla: Dan frente a una puerta cerrada y un indicador de llave.

Narración: «Dan llegó a un castillo. La puerta solo se abre si tiene una llave. ¿Cómo hacemos para que el juego decida si puede pasar?».

Escena 2 · Dos caminos (0:30–2:30)

Pantalla: diagrama: «¿tiene llave?» → Sí: abrir / No: buscar llave; al lado, código.

Narración: «En Java usamos if, que significa “si”. Si la condición es verdadera, ejecuta las instrucciones entre llaves. Si es falsa, podemos usar else, que significa “si no”. Primero probamos con tieneLlave = false. ¿Qué mensaje aparecerá? [Pausa]. Ahora cambiemos a true y ejecutemos otra vez La condición se escribe entre paréntesis y las acciones van entre llaves. No estamos preguntándole al jugador: Java comprueba el valor que ya está guardado en tieneLlave.».

Escena 3 · Tres resultados posibles (2:30–4:30)

Pantalla: barra de energía con niveles 80, 35 y 0.

Narración: «No todas las decisiones tienen solo dos opciones. Podemos usar else if para revisar otra condición. Si Dan tiene 60 o más de energía, está listo para avanzar. Si tiene algo de energía, pero menos de 60, necesita descansar. Y si llega a cero, termina la partida. Java revisa estas preguntas en orden y usa la primera que sea verdadera El orden importa: primero comprobamos si tiene al menos 60; si no, comprobamos si aún le queda algo. else se ejecuta cuando ninguna condición anterior se cumple.».

Escena 4 · Errores comunes (4:30–5:30)

Pantalla: `=` frente a `==`; luego `zona.equals("bosque")`.

Narración: «Recuerda: = sirve para asignar un valor; == compara. Si comparamos textos, como el nombre de una zona, usamos equals. Por ejemplo, zona.equals("bosque") pregunta si el texto es “bosque” Por ejemplo, energia = 35 guarda 35, mientras que energia == 35 pregunta si la energía es 35. Para comparar el contenido de dos textos, usamos equals() en lugar de ==.».

Escena 5 · Práctica (5:30–6:00)

Pantalla: «Prueba energía 80, 35 y 0».

Narración: «Cambia la energía y predice el mensaje antes de ejecutar. En la próxima lección repetiremos acciones sin copiar la misma instrucción muchas veces».

Código de demostración

```
public class Main {
    public static void main(String[] args) {
        boolean tieneLlave = false;
        int energia = 35;

        if (tieneLlave) {
            System.out.println("¡La puerta se abre!");
        } else {
            System.out.println("Busca la llave del castillo.");
```

```
        }

        if (energia >= 60) {
            System.out.println("Dan está listo para avanzar.");
        } else if (energia > 0) {
            System.out.println("Dan necesita descansar.");
        } else {
            System.out.println("Fin de la partida.");
        }

        String zona = "bosque";
        if (zona.equals("bosque")) {
            System.out.println("Has entrado al bosque.");
        }
    }
}
```

Práctica principal: prueba energía 80, 35 y 0; explica qué cambia.

Desafíos opcionales: abre la puerta cambiando `tieneLlave`; añade un mensaje si `monedas >= 10`; detecta un `=` escrito donde debería ir `==`.

Video 5 — Repetir tareas con bucles (6 min)

Objetivo: usar `for` y `while` para repetir instrucciones.

Idea clave: los bucles permiten repetir rondas y acciones sin duplicar código.

Guion por escenas

Escena 1 · Cinco rondas (0:00–0:30)

Pantalla: cinco casillas de un camino; Dan debe pasar por todas.

Narración: «Dan debe completar cinco rondas para llegar al castillo. ¿Escribiríamos cinco veces el mismo mensaje? Hay una forma más sencilla: usar un bucle».

Escena 2 · `for`: contar rondas (0:30–2:30)

Pantalla: colorear inicio `int ronda = 1`, condición `ronda <= 5`, avance `ronda++`.

Narración: «Un bucle for sirve cuando sabemos cuántas veces queremos repetir algo. Empezamos en la ronda uno, continuamos mientras el número sea cinco o menor y sumamos uno después de cada vuelta. Observa cómo el contador cambia de uno a cinco. Antes de ejecutar, ¿cuál será el último número mostrado? En for (int ronda = 1; ronda <= 5; ronda++), la primera parte crea el contador, la segunda indica cuándo continuar y ronda++ aumenta el contador en uno. Cada repetición se llama iteración.».

Escena 3 · Puntos por ronda (2:30–3:45)

Pantalla: cada ronda entrega 10 puntos.

Narración: «Ahora el juego entrega diez puntos por ronda. Usamos el número de ronda para calcular cuántos puntos acumuló Dan. No necesitamos escribir una línea diferente para cada ronda Si la ronda vale 3, la expresión ronda * 10 da 30. Los paréntesis hacen que Java calcule esa multiplicación antes de unir el resultado con el mensaje.».

Escena 4 · `while`: cuenta regresiva (3:45–5:00)

Pantalla: puerta que se abre tras contar 3, 2, 1.

Narración: «El bucle while significa “mientras”. Repite instrucciones mientras una condición sea verdadera. Vamos a hacer una cuenta regresiva para abrir la puerta. Restamos uno en cada vuelta. Si olvidamos restar, la condición puede seguir siendo verdadera para siempre: eso se llama bucle infinito. En ese caso, detenemos el programa y revisamos el código En cuenta--, los dos signos menos significan restar uno. Sin ese cambio, cuenta seguiría valiendo 3 y la cuenta regresiva nunca terminaría.».

Escena 5 · Práctica (5:00–6:00)

Pantalla: «Cambia las rondas de 5 a 3».

Narración: «Prueba haciendo que Dan complete solo tres rondas. ¿Cuántos puntos consigue? En el siguiente video crearemos acciones que podremos llamar cada vez que las necesitemos».

Código de demostración

```
public class Main {
    public static void main(String[] args) {
        for (int ronda = 1; ronda <= 5; ronda++) {
            System.out.println("Ronda " + ronda + " completada");
            System.out.println("Puntos acumulados: " + (ronda * 10));
        }

        int cuenta = 3;
        while (cuenta > 0) {
```

```
            System.out.println("La puerta se abre en " + cuenta);
            cuenta--;
        }
        System.out.println("¡Puerta abierta!");
    }
}
```

Práctica principal: cambia el `for` para mostrar tres rondas.

Desafíos opcionales: aumenta los puntos por ronda; haz una cuenta regresiva de cinco; muestra solo las rondas pares.

Video 6 — Métodos y parámetros (6 min)

Objetivo: crear métodos con parámetros y valores de retorno.

Idea clave: un método es una acción con nombre que podemos reutilizar.

Guion por escenas

Escena 1 · Los poderes del héroe (0:00–0:30)

Pantalla: botones imaginarios «Saludar», «Curar», «Calcular puntos».

Narración: «En nuestro juego, Dan necesita realizar acciones varias veces. Por ejemplo, saludar a un nuevo personaje o recuperar energía. ¿Podemos crear una acción una sola vez y utilizarla cuando queramos? Sí: con métodos».

Escena 2 · Primera acción (0:30–2:00)

Pantalla: resaltar método `saludar` y su llamada desde `main`.

Narración: «Un método es un bloque de instrucciones con nombre. Creamos saludar y luego lo llamamos desde main. La palabra void indica que este método realiza una acción, pero no devuelve un valor. Cuando escribimos saludar("Dan"), Java ejecuta el saludo Para reconocerlo, mira sus partes: public static void saludar(String nombre). Por ahora, céntrate en saludar, que es el nombre del método; void indica que no devuelve un valor. Los paréntesis permiten recibir información.».

Escena 3 · Parámetros (2:00–3:30)

Pantalla: el mismo botón saluda a Dan, Luna y Rocky con distintos nombres.

Narración: «¿Y si queremos saludar a Luna y a Rocky? No necesitamos crear tres métodos. Podemos pasarle un dato llamado parámetro. Aquí el parámetro es String nombre: un texto que el método recibe. El mismo método usa un nombre distinto cada vez que lo llamamos Cuando llamamos saludar("Luna"), Luna es el argumento: el dato concreto que enviamos. Dentro del método, ese dato se recibe en el parámetro nombre. Así podemos reutilizar las mismas instrucciones.».

Escena 4 · Métodos que devuelven resultados (3:30–5:00)

Pantalla: Dan recibe energía; flechas: energía actual + curación → energía nueva.

Narración: «Algunos métodos devuelven un resultado. recuperarEnergia recibe la energía actual y cuánto queremos recuperar. La palabra return entrega el nuevo valor. Como devuelve un número entero, escribimos int en lugar de void. Podemos guardar ese resultado en una variable En int recuperarEnergia(int actual, int cantidad), int antes del nombre indica el tipo de resultado que devolverá. return actual + cantidad calcula y entrega ese resultado; después lo guardamos en energia.».

Escena 5 · Práctica (5:00–6:00)

Pantalla: «Haz que Dan recupere 20 puntos de energía».

Narración: «Cambia la cantidad de energía recuperada y comprueba el resultado. Ya sabemos crear acciones reutilizables. En la próxima lección el jugador podrá escribir respuestas en nuestra aventura».

Código de demostración

```
public class Main {
    public static void saludar(String nombre) {
        System.out.println("¡Bienvenido, " + nombre + "!");
    }
```

```
    public static int recuperarEnergia(int actual, int cantidad) {
        return actual + cantidad;
    }

    public static boolean puedeAvanzar(int energia) {
        return energia > 0;
    }

    public static void main(String[] args) {
        saludar("Dan");
        saludar("Luna");

        int energia = recuperarEnergia(30, 20);
        System.out.println("Nueva energía: " + energia);
        System.out.println("¿Puede avanzar? " + puedeAvanzar(energia));
    }
}
```

Práctica principal: cambia `recuperarEnergia(30, 20)` para recuperar 10 puntos.

Desafíos opcionales: crea un método `ganarPuntos`; saluda a otro personaje; crea un método que devuelva si la energía es mayor o igual que 50.

Video 7 — Entrada y salida con `Scanner` (7 min)

Objetivo: recibir texto y números escritos por el jugador.

Idea clave: `Scanner` permite que el programa lea lo que escribimos en la consola.

Guion por escenas

Escena 1 · El jugador toma el control (0:00–0:40)

Pantalla: la aventura pregunta «¿Cómo se llama tu héroe?».

Narración: «Hasta ahora nosotros elegíamos los datos escribiéndolos en el código. Pero ¿qué pasa si queremos que cada jugador escriba su propio nombre y tome decisiones? Hoy haremos que nuestra aventura escuche sus respuestas».

Escena 2 · Conocer `Scanner` (0:40–2:00)

Pantalla: resaltar `import java.util.Scanner;`, `Scanner sc = new Scanner(System.in);`.

Narración: «Java incluye herramientas que podemos usar en nuestros programas. Una de ellas se llama Scanner y sirve para leer datos. La importamos al inicio del archivo y creamos un Scanner que leerá lo que el jugador escribe en la consola. No tienes que memorizar estas líneas: puedes usar el ejemplo precargado La palabra import indica que queremos utilizar una herramienta disponible en Java. System.in representa la entrada del teclado, es decir, lo que escribe la persona que juega.».

Escena 3 · Nombre y respuesta (2:00–4:00)

Pantalla: escribir nombre; mostrar bienvenida; explicar `nextLine` y `nextInt`.

Narración: «Con nextLine() leemos una línea de texto completa, como el nombre del héroe. Con nextInt() leemos un número entero, como una opción del menú. print muestra la pregunta sin saltar a la línea siguiente, mientras que println sí salta. Probemos: escribo “Dan” y la aventura me saluda La respuesta se guarda en una variable: String nombre = sc.nextLine() almacena texto, mientras que sc.nextInt() obtiene un entero. Así el juego puede usar lo que escribió el jugador en lugar de un dato fijo.».

Escena 4 · El cofre misterioso (4:00–6:00)

Pantalla: cofre con clave numérica del 1 al 10; ejecutar con intentos 3, 9 y 7.

Narración: «Ahora Dan encontró un cofre protegido por una clave del uno al diez. Para practicar, la clave será siete. Si escribimos tres, el juego dirá “Prueba un número más alto”. Si escribimos nueve, dirá “Prueba uno más bajo”. Con un while, seguirá preguntando hasta acertar. Observa cómo combinamos lo que ya aprendimos: variables, condiciones y bucles Cada intento se guarda en la variable intento. El if comprueba si el número es menor que la clave y el else if si es mayor; cuando ambos son falsos, significa que acertamos.».

Escena 5 · Práctica y error común (6:00–7:00)

Pantalla: «Adivina la clave»; nota pequeña: «`nextInt()` seguido de `nextLine()` puede requerir consumir el salto de línea».

Narración: «Prueba adivinando la clave. Un detalle: después de leer un número con nextInt(), puede quedar pendiente un salto de línea. Si luego quieres leer texto con nextLine(), puedes necesitar un nextLine() adicional. En el próximo video conoceremos a las mascotas de nuestra aventura Si ocurre ese problema, no es que Java haya perdido tu respuesta: quedó pendiente el salto de línea de la entrada anterior. Añadir sc.nextLine() después de nextInt() lo consume antes de pedir otro texto.».

Código de demostración

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nombre de tu héroe: ");
        String nombre = sc.nextLine();
        System.out.println("¡Bienvenido a la aventura, " + nombre + "!");

        int clave = 7;
        int intento = 0;
        while (intento != clave) {
            System.out.print("Adivina la clave del cofre (1 a 10): ");
            intento = sc.nextInt();

            if (intento < clave) {
                System.out.println("Prueba un número más alto.");
            } else if (intento > clave) {
                System.out.println("Prueba un número más bajo.");
            }
        }
        System.out.println("¡Abriste el cofre!");
    }
}
```

Práctica principal: ejecuta el juego y abre el cofre con pistas.

Desafíos opcionales: cambia la clave; cuenta cuántos intentos usó el jugador; pregunta cuántas vidas desea usar y muestra la respuesta.

Video 8 — Clases, objetos y constructores (7 min)

Objetivo: comprender clases, atributos, métodos, objetos y constructores.

Idea clave: una clase describe un tipo de personaje; los objetos son personajes concretos creados con ella.

Guion por escenas

Escena 1 · Nuevos compañeros de aventura (0:00–0:40)

Pantalla: Dan encuentra dos mascotas: Luna, una gata, y Rocky, un perro.

Narración: «Dan ya sabe abrir cofres, pero ahora necesita compañeros de aventura. Conozcamos a Luna y Rocky. Los dos tienen nombre, tipo y energía; ambos pueden jugar y dormir. ¿Tendremos que escribir un programa completamente distinto para cada mascota? No. Podemos crear una plantilla».

Escena 2 · Clase y objetos (0:40–2:15)

Pantalla: tarjeta `Mascota` → flechas hacia Luna y Rocky; resaltar «atributos» y «métodos».

Narración: «En Java, esa plantilla se llama clase. Describe las características que tendrán las mascotas, como nombre, tipo y energía. Esas características se llaman atributos. También describe acciones, como jugar y dormir, que se llaman métodos. Luna y Rocky son objetos: mascotas concretas creadas con la misma clase, pero cada una con sus propios datos. ¿Qué pasará con la energía de Rocky si solo juega Luna? [Pausa]. No cambia Piensa en Mascota como el diseño general, y en Luna como un ejemplar creado a partir de ese diseño. La clase describe qué datos y acciones tendrán; cada objeto conserva sus propios valores.».

Escena 3 · Construir la clase (2:15–4:30)

Pantalla: mostrar atributos, luego constructor y finalmente acciones.

Narración: «Escribimos la clase Mascota. Después agregamos un constructor, un bloque especial que prepara cada mascota al crearla. Tiene el mismo nombre que la clase y recibe el nombre y el tipo de la mascota. this.nombre significa “el nombre de esta mascota”. Cada mascota comienza con 50 de energía. Después escribimos sus acciones: jugar gasta 10 de energía y dormir recupera 20, sin superar 100 El constructor no es un método común: es una parte especial que se ejecuta al crear un objeto con new. La palabra this ayuda a distinguir el atributo del objeto del dato recibido como parámetro.».

Escena 4 · Crear mascotas (4:30–6:15)

Pantalla: resaltar `new Mascota(...)`, luego `luna.jugar()` y `rocky.dormir()`.

Narración: «Para crear un objeto usamos new y llamamos al constructor. Así nacen Luna y Rocky. Cuando escribimos luna.jugar(), Luna pierde energía. Rocky conserva la suya hasta que ejecutamos una acción sobre él. Esto ocurre porque cada objeto guarda sus propios datos En Mascota luna = new Mascota("Luna", "gata");, Mascota es el tipo, luna es la variable y new crea el objeto. El punto en luna.jugar() nos permite llamar a una acción de esa mascota concreta.».

Escena 5 · Práctica (6:15–7:00)

Pantalla: «Crea una tercera mascota: Nube».

Narración: «Pausa y crea una mascota llamada Nube. Haz que juegue una vez. En el próximo video aprenderemos a evitar que alguien cambie la energía de una mascota a valores imposibles».

Código de demostración

```
class Mascota {
    String nombre;
    String tipo;
    int energia;

    Mascota(String nombre, String tipo) {
        this.nombre = nombre;
        this.tipo = tipo;
        this.energia = 50;
    }

    void jugar() {
        energia = Math.max(0, energia - 10);
        System.out.println(nombre + " juega. Energía: " + energia);
    }

    void dormir() {
        energia = Math.min(100, energia + 20);
        System.out.println(nombre + " duerme. Energía: " + energia);
    }
}

public class Main {
    public static void main(String[] args) {
        Mascota luna = new Mascota("Luna", "gata");
        Mascota rocky = new Mascota("Rocky", "perro");
        luna.jugar();
        luna.jugar();
        rocky.dormir();
    }
}
```

Nota para narración: `Math.max` y `Math.min` mantienen la energía entre 0 y 100; no es necesario explicar todavía su funcionamiento interno.

Práctica principal: crea `Mascota nube = new Mascota("Nube", "conejo");` y llama a `nube.jugar();`.

Desafíos opcionales: predice la energía de Luna y Rocky; crea otra mascota; agrega una acción `mostrarEstado()`.

Video 9 — Encapsulación y arreglos con objetos (6 min)

Objetivo: proteger atributos con `private`, consultar con getters, modificar con setters y recorrer arreglos de objetos.

Idea clave: los objetos pueden proteger sus datos y podemos agrupar varios en un arreglo.

Guion por escenas

Escena 1 · ¿Energía infinita? (0:00–0:40)

Pantalla: Luna con una barra de energía de 50; alguien intenta cambiarla a 500.

Narración: «¡Tenemos un problema en nuestro juego! ¿Qué pasaría si alguien pudiera poner la energía de Luna en quinientos, cuando el máximo es cien? Necesitamos reglas para cuidar los datos de nuestras mascotas».

Escena 2 · `private`, getter y setter (0:40–3:00)

Pantalla: candado sobre `energia`; botones «consultar energía» y «cambiar energía».

Narración: «Con private impedimos que otras clases cambien directamente la energía. Para consultarla usamos un método llamado getter: getEnergia(). Para intentar cambiarla usamos un setter: setEnergia(...). El setter comprueba que el número esté entre cero y cien. Probemos con 80 y con 500. ¿Cuál crees que se aceptará? [Pausa]. Esto es una forma sencilla de encapsulación: proteger datos y controlar cómo se modifican private significa que el atributo no puede modificarse directamente desde otra clase. getEnergia devuelve su valor actual; setEnergia recibe un valor nuevo y decide si es válido antes de guardarlo.».

Escena 3 · Un equipo de mascotas (3:00–4:45)

Pantalla: tres casillas numeradas 0, 1, 2 con Luna, Rocky y Nube.

Narración: «Ahora Dan tiene tres mascotas. En lugar de crear una lista de mensajes por separado, podemos guardar sus objetos en un arreglo. Piensa en una fila de casillas que comienza en la posición cero. length nos indica cuántas casillas hay, y un for nos permite visitarlas una por una. Así mostramos los nombres de todas las mascotas El arreglo Mascota[] equipo guarda varias mascotas del mismo tipo. equipo[0] es la primera, equipo[1] la segunda y equipo[2] la tercera. El for recorre esas posiciones sin tener que escribir tres instrucciones diferentes.».

Escena 4 · Una herramienta para los nombres (4:45–5:30)

Pantalla: `"Luna".toUpperCase()` → `LUNA`; `"Luna".equals("Luna")` → `true`.

Narración: «Los textos también tienen métodos. toUpperCase() convierte un nombre a mayúsculas y equals() permite comparar textos. Los veremos rápidamente, porque nuestra práctica principal sigue siendo proteger la energía y recorrer el equipo Observa los paréntesis al final de toUpperCase() y equals(): nos indican que estamos llamando métodos del texto. equals() devuelve true si ambos textos tienen el mismo contenido.».

Escena 5 · Práctica (5:30–6:00)

Pantalla: «Prueba 80 y 500; muestra las tres mascotas».

Narración: «Cambia la energía de Luna a 80 y luego intenta poner 500. ¿Qué sucede? En la última lección reuniremos estas ideas en una aventura de texto que podrás jugar».

Código de demostración

```
class Mascota {
    private String nombre;
```

```
    private int energia;

    Mascota(String nombre) {
        this.nombre = nombre;
        this.energia = 50;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEnergia() {
        return energia;
    }

    public void setEnergia(int nuevaEnergia) {
        if (nuevaEnergia >= 0 && nuevaEnergia <= 100) {
            energia = nuevaEnergia;
        } else {
            System.out.println("Energía inválida: usa un valor de 0 a 100.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Mascota luna = new Mascota("Luna");
        luna.setEnergia(80);
        luna.setEnergia(500);
        System.out.println(luna.getNombre() + ": " + luna.getEnergia());

        Mascota[] equipo = {
            luna, new Mascota("Rocky"), new Mascota("Nube")
        };
        for (int i = 0; i < equipo.length; i++) {
            System.out.println("Mascota " + (i + 1) + ": " + equipo[i].getNombre());
        }

        System.out.println("LUNA: " + luna.getNombre().toUpperCase());
        System.out.println("¿Se llama Luna? " + luna.getNombre().equals("Luna"));
    }
}
```

Práctica principal: prueba `setEnergia(80)` y `setEnergia(500)` y explica qué sucede.

Desafíos opcionales: añade una cuarta mascota; muestra solo mascotas con energía mayor que 40; cambia el nombre mostrado a mayúsculas.

Video 10 — Proyecto final y buenas prácticas (8 min)

Objetivo: integrar variables, operadores, condiciones, bucles, métodos, entrada/salida, clases, encapsulación y arreglos en una aventura sencilla.

Idea clave: con pequeñas piezas de código podemos crear un programa interactivo completo.

Guion por escenas

Escena 1 · La aventura está lista (0:00–0:40)

Pantalla: línea de progreso: mensaje → ficha → puntos → decisiones → rondas → acciones → respuestas → mascotas → equipo.

Narración: «Durante nueve videos construimos las piezas de una aventura: aprendimos a guardar datos, tomar decisiones, repetir acciones y crear mascotas. Ahora vamos a unirlas en un pequeño juego de texto. ¡No necesitas escribir todo desde cero! El proyecto inicial estará preparado en el enlace de la descripción».

Escena 2 · ¿Cómo se juega? (0:40–1:40)

Pantalla: menú «1. Explorar / 2. Ver equipo / 3. Descansar / 4. Salir».

Narración: «Primero elegimos el nombre de nuestro héroe. Luego aparece un menú. Explorar nos da puntos, pero gasta energía. Ver equipo muestra nuestras mascotas. Descansar recupera energía. Salir termina la partida. Vamos a completar juntos algunas partes del programa y probarlo Cada número del menú representa una acción. El programa no necesita gráficos para ser interactivo: recibe una elección y muestra una respuesta distinta.».

Escena 3 · Clase y equipo (1:40–3:15)

Pantalla: destacar `Mascota`, `private`, constructor, getter y arreglo `equipo`.

Narración: «Aquí está nuestra clase Mascota, que ya conocemos. Tiene un nombre protegido, un constructor y un método para consultar ese nombre. Creamos a Luna y Rocky y los guardamos en un arreglo llamado equipo. No necesitamos memorizar cada línea: lo importante es reconocer para qué sirve cada pieza La palabra private protege el nombre de la mascota; getNombre() permite consultarlo. El arreglo reúne varios objetos Mascota para que podamos mostrarlos como un equipo.».

Escena 4 · Menú y acciones (3:15–5:45)

Pantalla: mostrar `Scanner`, `while`, `if-else`, método `mostrarEquipo` y cambios en puntos y energía.

Narración: «Con Scanner recibimos la opción que escribe el jugador. El while mantiene el menú activo hasta elegir salir. Si exploramos y tenemos energía suficiente, ganamos diez puntos y gastamos diez de energía. Si no tenemos suficiente, el juego nos recomienda descansar. La opción dos llama al método mostrarEquipo, que recorre el arreglo de mascotas. Probemos una exploración y después revisemos el equipo Scanner lee el número elegido; if y else if comprueban qué opción corresponde. La instrucción puntos += 10 suma diez al valor anterior, y energia -= 10 resta diez. Son formas abreviadas de escribir puntos = puntos + 10 y energia = energia - 10.».

Escena 5 · Buenas prácticas y errores comunes (5:45–7:15)

Pantalla: antes/después de nombres claros, sangría, `;`, `==`, `equals`, límites del arreglo.

Narración: «Al programar, usa nombres que expliquen para qué sirve cada dato, como energia o puntos. Mantén las instrucciones ordenadas con sangría y prueba tu código después de pequeños cambios. Si aparece un error, revisa el mensaje y la línea indicada. Entre los errores más comunes están olvidar el punto y coma, confundir = con ==, comparar textos sin equals() o

intentar acceder a una posición que no existe en un arreglo. Equivocarse también es parte de aprender Si el programa no se ejecuta, comprueba primero los signos y las llaves. Si sí se ejecuta pero muestra algo inesperado, revisa los valores de las variables y prueba una opción a la vez.».

Escena 6 · Final y desafío (7:15–8:00)

Pantalla: «¡Tu aventura funciona!» y reto «Agrega una tercera mascota».

Narración: «¡Felicidades! Ya tienes una aventura que responde a tus decisiones. Ahora puedes cambiar nombres, agregar una mascota o inventar una nueva opción del menú. Lo importante no es que el juego sea enorme, sino que entiendas cómo se construyó paso a paso. ¡Tu próxima aventura empieza con una nueva idea!».

Código completo de referencia (precargado antes del video)

> **Producción:** no escribir todo este programa durante la grabación. Mostrar el archivo precargado y completar o modificar solo tres fragmentos: una opción del menú, la exploración y la visualización del equipo. Entregar un archivo inicial con espacios marcados y este archivo completo en el repositorio.

```
import java.util.Scanner;

class Mascota {
    private String nombre;

    Mascota(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }
}

public class Main {
    public static void mostrarEquipo(Mascota[] equipo) {
        System.out.println("Tus compañeros:");
        for (int i = 0; i < equipo.length; i++) {
            System.out.println((i + 1) + ". " + equipo[i].getNombre());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("¿Cómo se llama tu héroe? ");
        String nombre = sc.nextLine();

        int puntos = 0;
        int energia = 30;
        int opcion = 0;
        Mascota[] equipo = {
            new Mascota("Luna"), new Mascota("Rocky")
        };

        System.out.println("¡Bienvenido, " + nombre + "!");
        while (opcion != 4) {
            System.out.println("\nEnergía: " + energia + " | Puntos: " + puntos);
            System.out.println("1. Explorar");
            System.out.println("2. Ver equipo");
```

```
            System.out.println("3. Descansar");
            System.out.println("4. Salir");
            System.out.print("Elige una opción: ");
            opcion = sc.nextInt();

            if (opcion == 1) {
                if (energia >= 10) {
                    energia -= 10;
                    puntos += 10;
                    System.out.println("¡Exploraste y ganaste 10 puntos!");
                } else {
                    System.out.println("No tienes energía suficiente. Descansa.");
                }
            } else if (opcion == 2) {
                mostrarEquipo(equipo);
            } else if (opcion == 3) {
                energia = Math.min(100, energia + 20);
                System.out.println("¡Recuperaste energía!");
            } else if (opcion != 4) {
                System.out.println("Esa opción no existe.");
            }
        }
        System.out.println("¡Hasta la próxima, " + nombre + "!");
    }
}
```

Práctica principal: abre el proyecto precargado, explora una vez, revisa el equipo y descansa.

Desafíos opcionales: agrega a Nube al arreglo; cambia la recompensa de 10 a 15 puntos; añade una opción «Ver instrucciones»; muestra un mensaje especial cuando se alcancen 50 puntos.

Recursos y entregables pendientes

- **Plan de curso Markdown:** resumen, público, duración, descripción de cada video, enlaces reales a YouTube, enlaces precargados de OnlineGDB, repositorio y elaboración en equipo.
- **Repositorio público:** `java-fundamentals-course-[equipo]` con `README.md`, `/starter-files`, `/completed-examples` y `/scripts`.
- **Videos:** 10 videos de 5–8 minutos, subidos como no listados, con narración, anotaciones, texto legible y práctica.
- **Validación:** ejecutar los 10 ejemplos en OnlineGDB antes de grabar; probar los enlaces sin instalación y revisar la experiencia desde un navegador limpio.